

AIE683 Machine Learning

# **Week 5: Tree Ensembles: Trees, Random Forests & Gradient Boosting**

Dr. Hakan Emekci

Hacettepe University, Graduate School of Informatics

# A table, a decision

---

- A lender approves or rejects a loan; an insurer flags a claim; a hospital predicts readmission.
- The data: one row per person, a few dozen columns, mixed types: age, income, occupation, region, number of previous claims, missing entries.
- Today's running example: **UCI Adult census**, 48,842 people, 14 columns (8 categorical). Predict whether annual income exceeds \$50K.
- On data like this, a tuned gradient-boosted tree ensemble is the baseline every other method must beat.

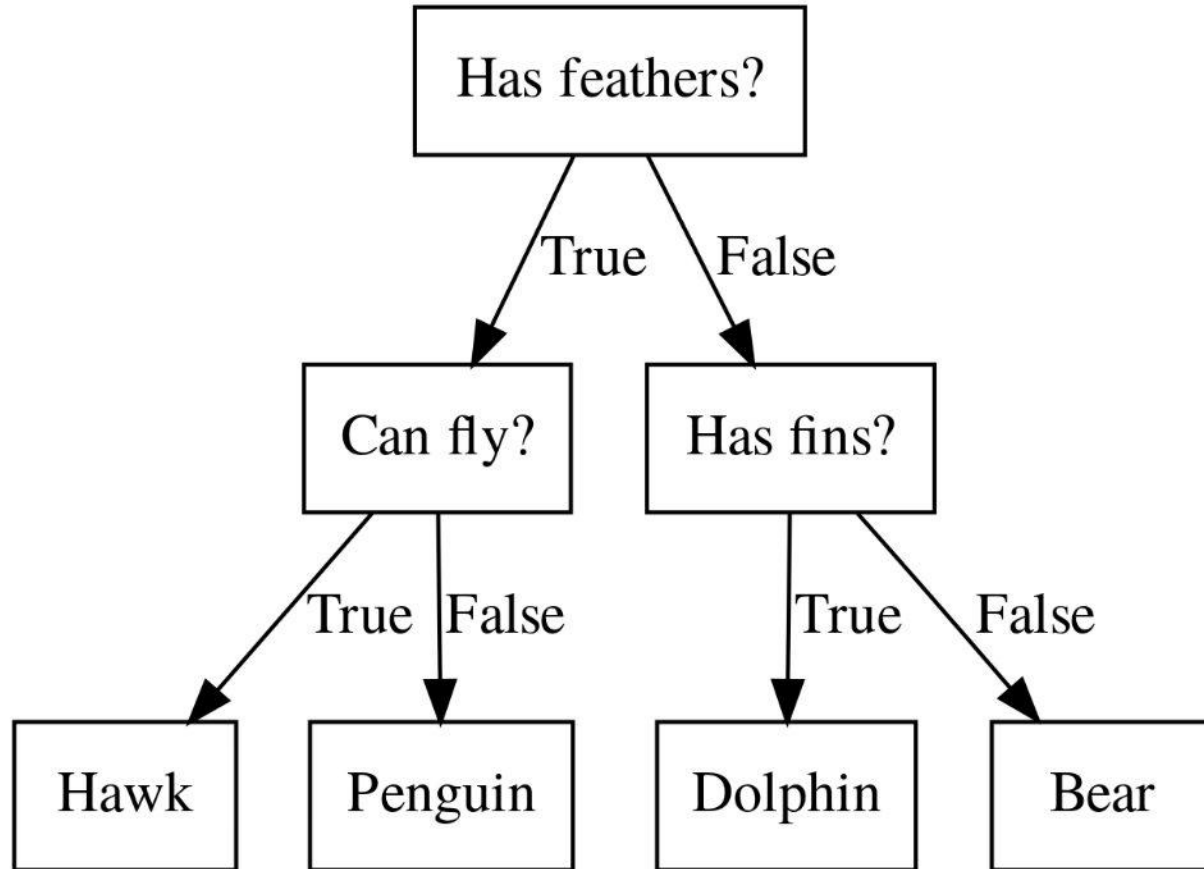
# Why Trees?

- Non-linear, and they model interactions automatically
- Insensitive to the scale and distribution of each feature: little preprocessing
- Small trees are interpretable
- Building block of the strongest off-the-shelf models for tabular data

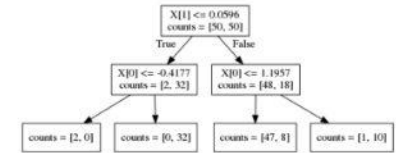
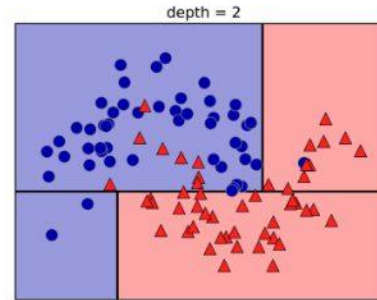
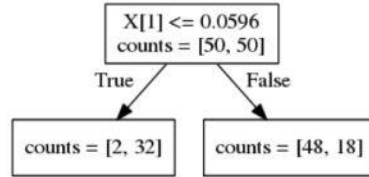
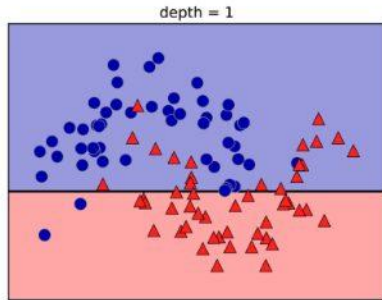
# Decision Trees for Classification

# Idea: series of binary questions

---

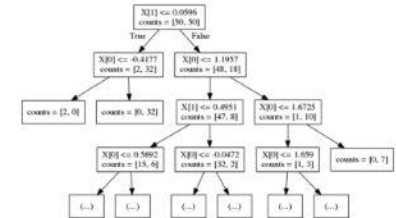
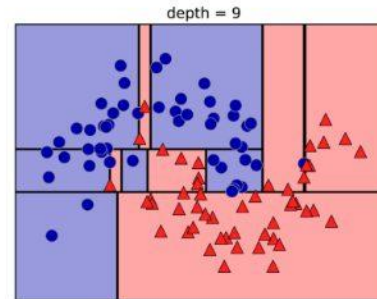


# Building Trees



Continuous features:

- "questions" are thresholds on single features.
- Minimize impurity



# Criteria (for classification)

---

- Gini Index:

$$H_{\text{gini}}(\mathbf{X}_m) = \sum_{k \in \square} p_{mk}(1 - p_{mk})$$

- Cross-Entropy:

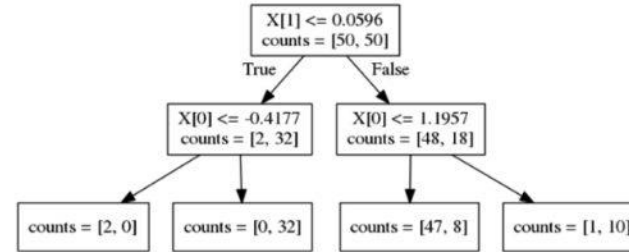
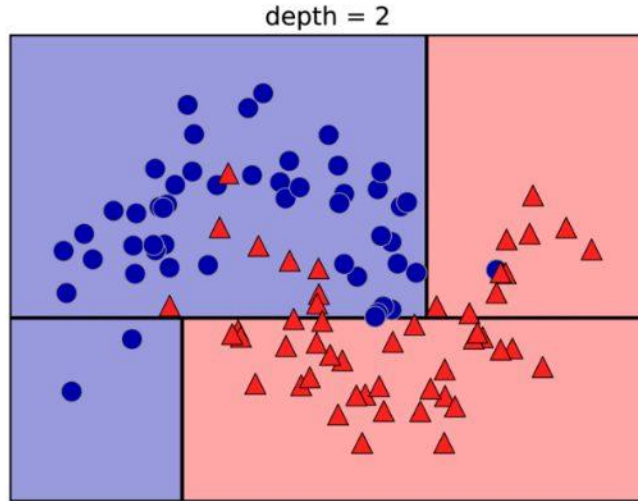
$$H_{\text{CE}}(\mathbf{X}_m) = - \sum_{k \in \square} p_{mk} \log(p_{mk})$$

$\mathbf{X}_m$  observations in node m

$\square$  classes

$p_m$  distribution over classes in node m

# Prediction



- Traverse tree based on feature tests
- Predict most common class in leaf



# Regression trees

---

$$\text{Prediction: } \bar{y}_m = \frac{1}{N_m} \sum_{i \in N_m} y_i$$

Mean Squared Error:

$$H(X_m) = \frac{1}{N_m} \sum_{i \in N_m} (y_i - \bar{y}_m)^2$$

Mean Absolute Error:

$$H(X_m) = \frac{1}{N_m} \sum_{i \in N_m} |y_i - \bar{y}_m|$$

- Without regularization / pruning:
- Each leaf often contains a single point to be "pure"

# Visualizing trees with sklearn

---

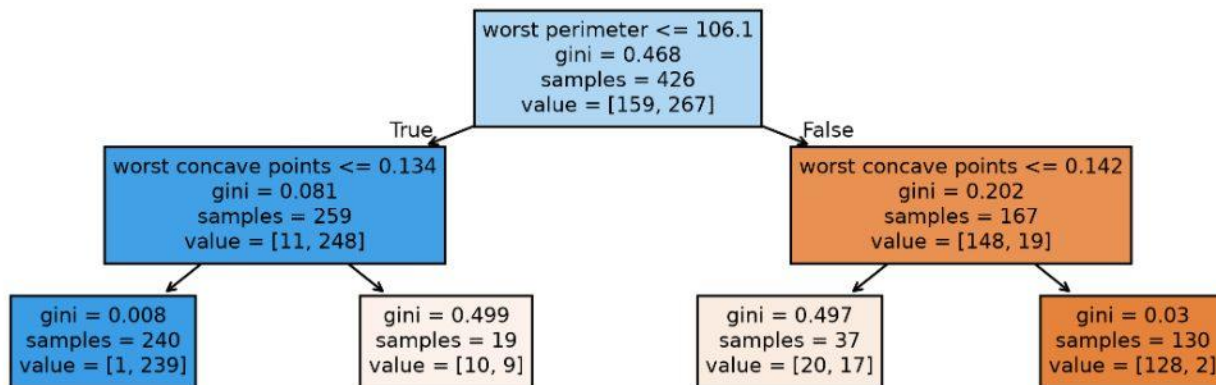
```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

cancer = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, stratify=cancer.target, random_state=0)

tree = DecisionTreeClassifier(max_depth=2)
tree.fit(X_train, y_train)
```

# Visualizing trees with sklearn

```
from sklearn.tree import plot_tree  
plot_tree(tree, feature_names=cancer.feature_names, filled=True)
```

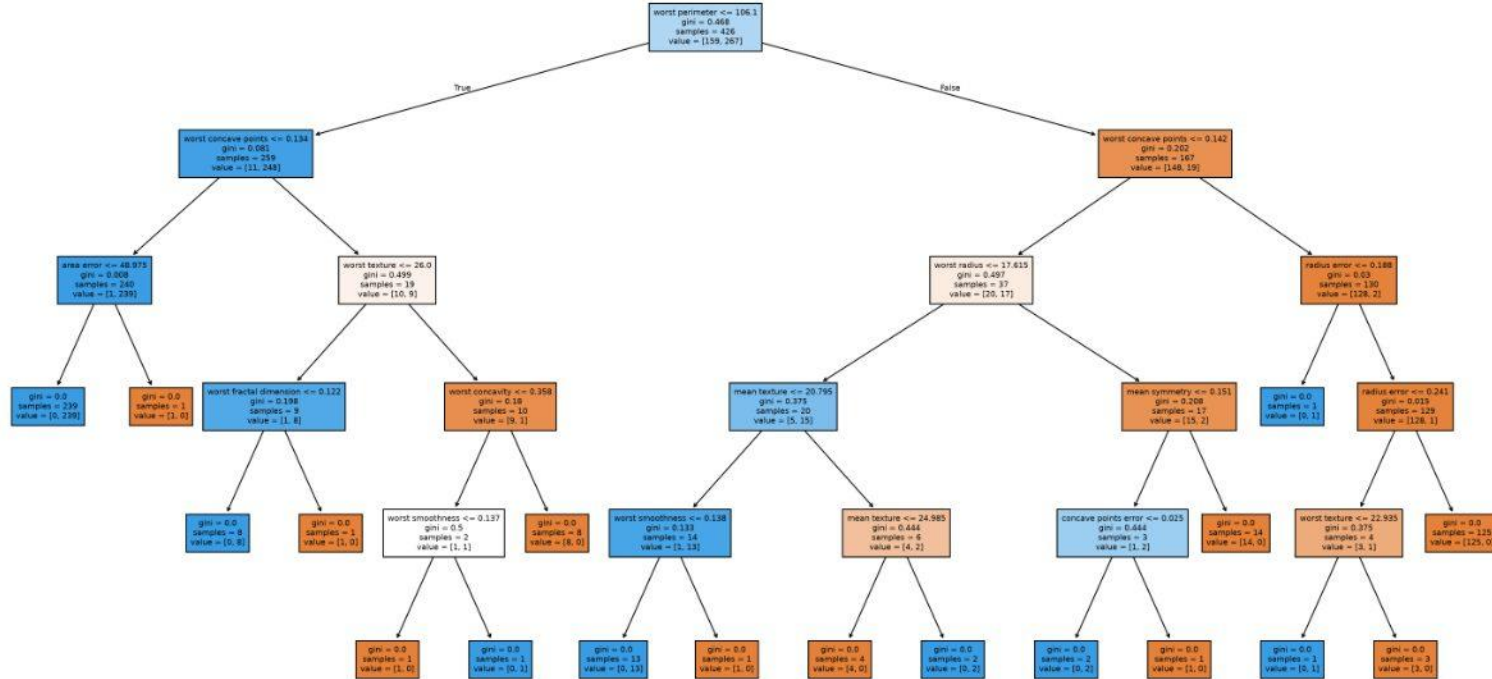


# Parameter Tuning

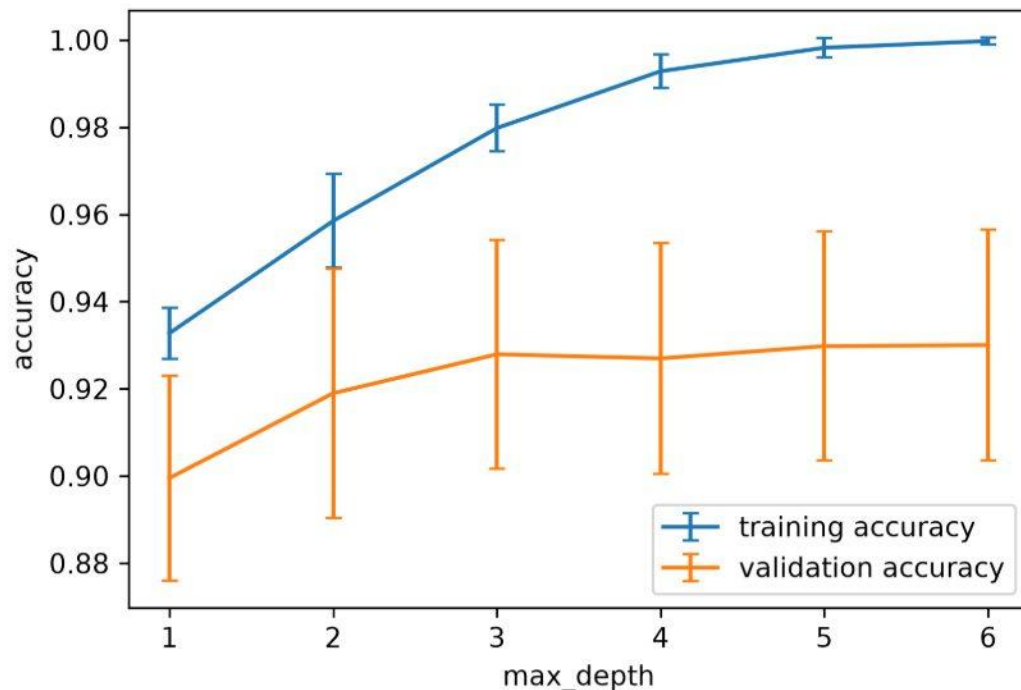
---

- Pre-pruning vs post-pruning
- Pre-pruning: Limit tree size (pick one, maybe two):
  - `max_depth`
  - `max_leaf_nodes`
  - `min_samples_split`
  - `min_impurity_decrease`
  - ...
- Post-pruning: grow the full tree, then cut back (`ccp_alpha`)

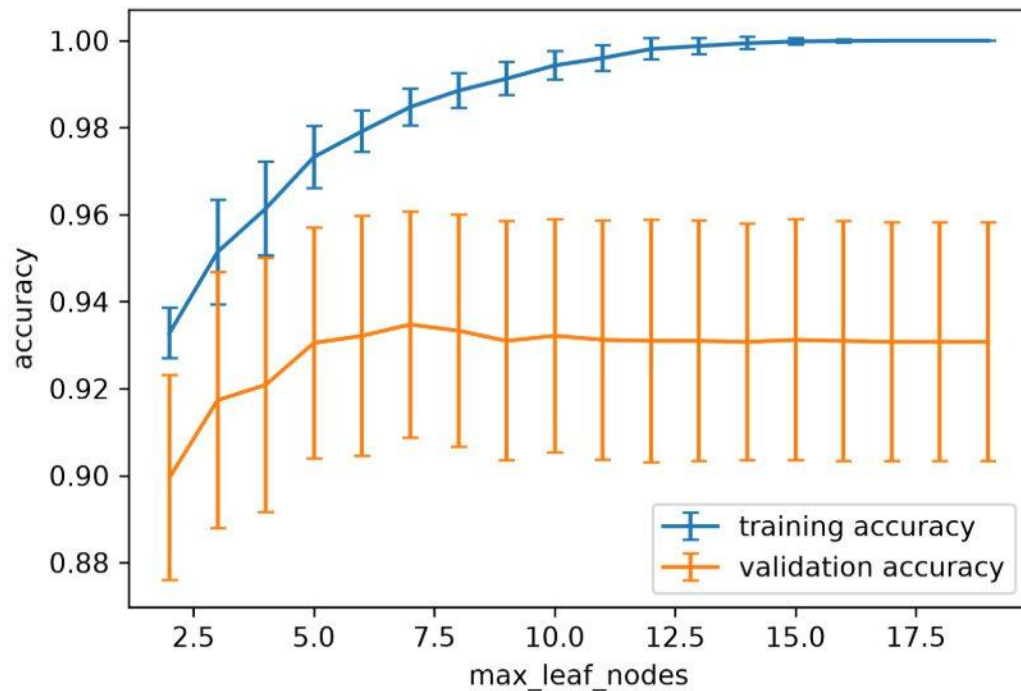
# No pruning



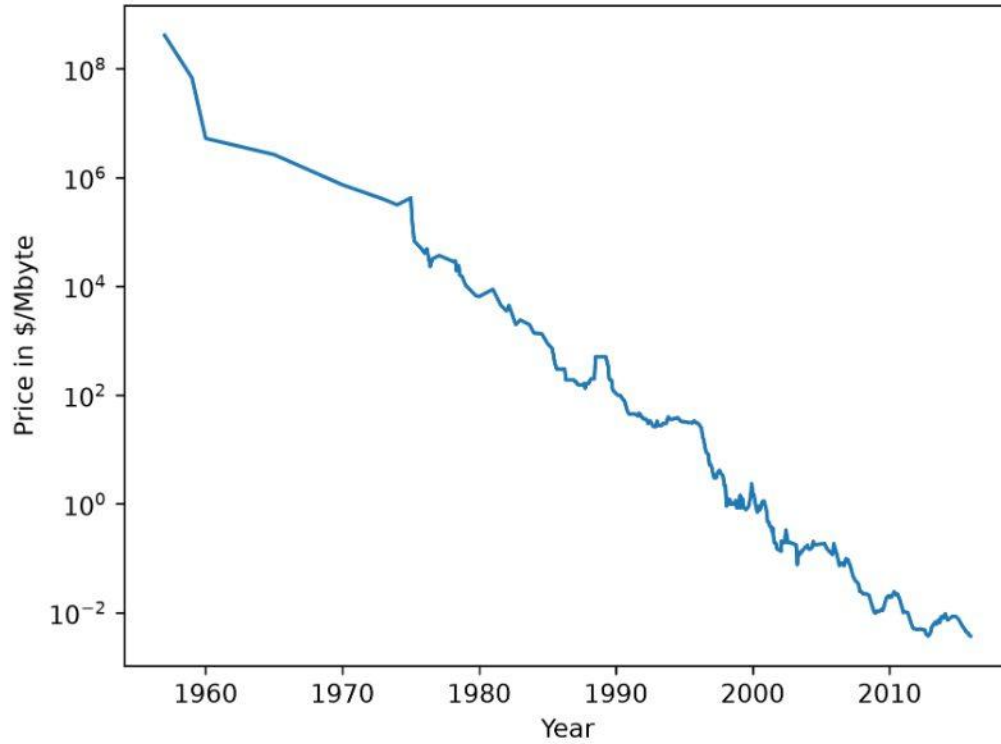
```
from sklearn.model_selection import GridSearchCV, RepeatedStratifiedKFold
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=10, random_state=0)
param_grid = {'max_depth': range(1, 7)}
grid = GridSearchCV(DecisionTreeClassifier(random_state=0),
                    param_grid=param_grid, cv=cv, return_train_score=True)
grid.fit(X_train, y_train)
```



```
param_grid = {'max_leaf_nodes': range(2, 20)}  
grid = GridSearchCV(DecisionTreeClassifier(random_state=0),  
                    param_grid=param_grid, cv=cv, return_train_score=True)  
grid.fit(X_train, y_train)
```

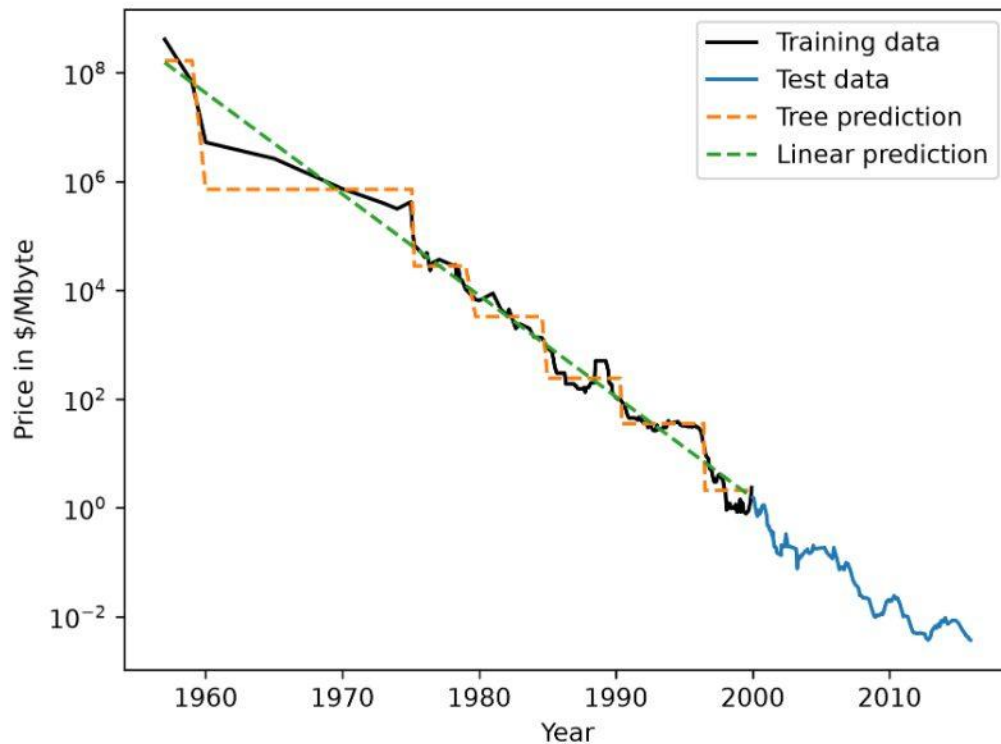


# Extrapolation

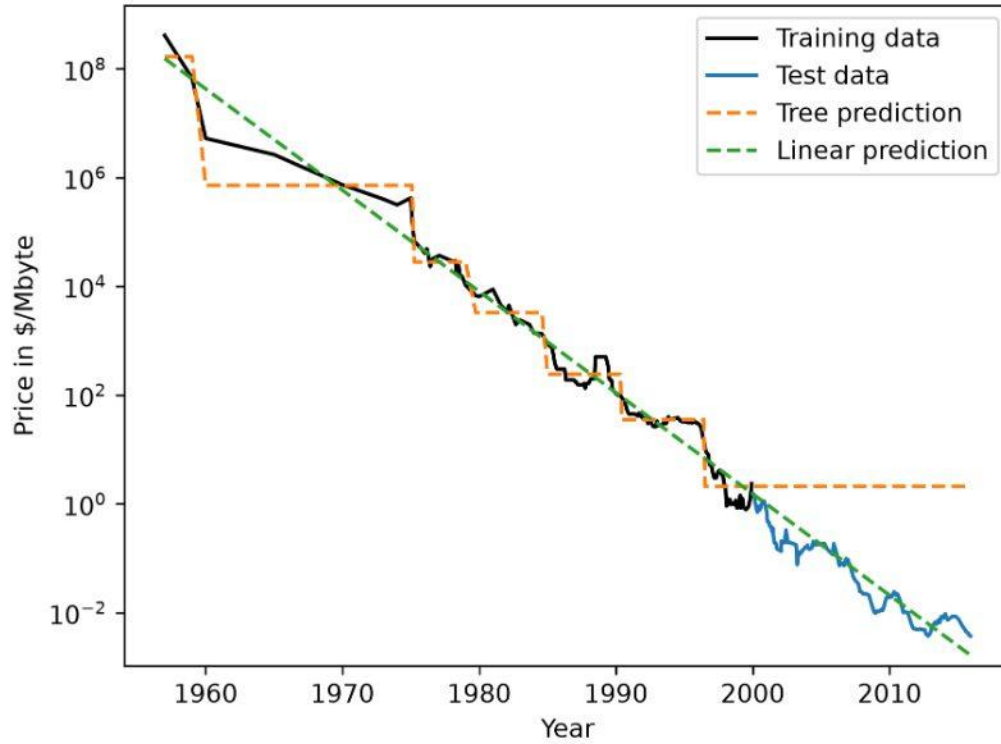




# Extrapolation

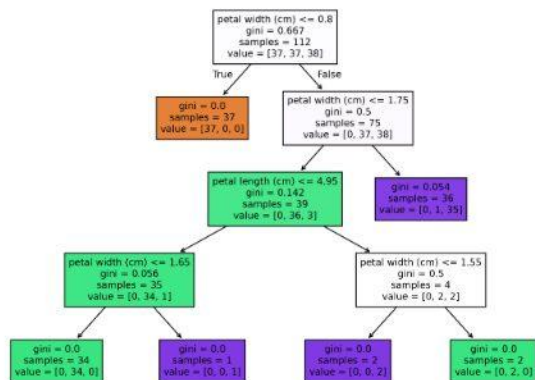


# Extrapolation

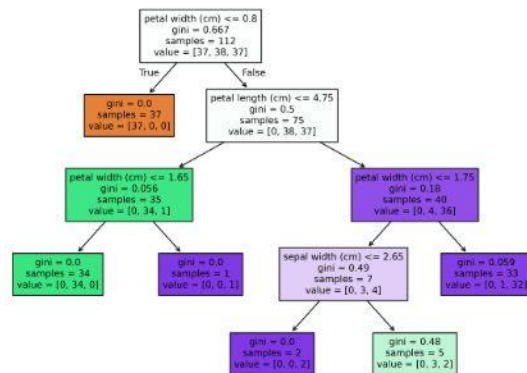


# Instability

```
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, stratify=iris.target, random_state=0)
tree = DecisionTreeClassifier(max_leaf_nodes=6, random_state=0)
tree.fit(X_train, y_train)
```

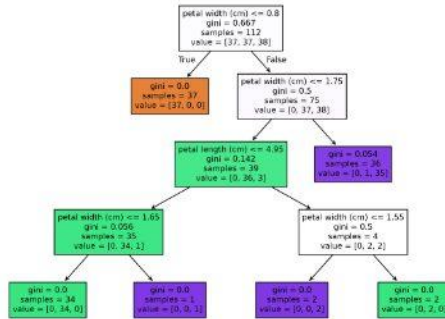


```
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, stratify=iris.target, random_state=13)
tree = DecisionTreeClassifier(max_leaf_nodes=6, random_state=0)
tree.fit(X_train, y_train)
```

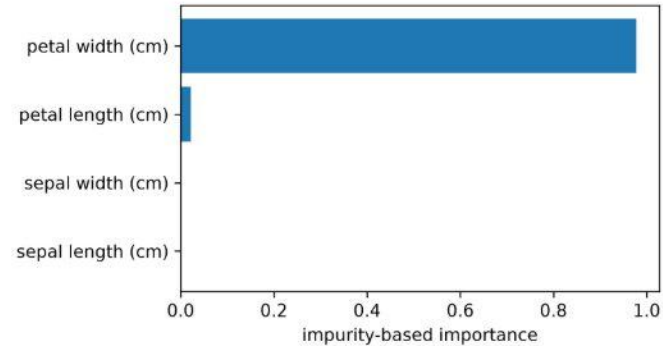


# Feature importance

```
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, stratify=iris.target, random_state=0)
tree = DecisionTreeClassifier(max_leaf_nodes=6, random_state=0)
tree.fit(X_train, y_train)
```



```
tree.feature_importances_  
array([0. , 0. , 0.022, 0.978])
```



- Unstable Tree → Unstable feature

importances.

- Might take one or multiple from a group of correlated features.

# Categorical Data and Missing Values

---

- A tree can split a categorical feature into two subsets of categories
- $2^{k-1} - 1$  possible splits for  $k$  categories, but the optimal one is found in  $O(k \log k)$  for binary classification (sort categories by mean target)
- scikit-learn `DecisionTreeClassifier` / `RandomForestClassifier`: encode first (ordinal encoding works well for trees)
- Native categorical splits: `HistGradientBoosting*`, `LightGBM`, `XGBoost`, `CatBoost` (Part 2)
- Missing values (`NaN`): supported natively by scikit-learn trees, forests and `HistGradientBoosting*`, and by `XGBoost`/`LightGBM`/`CatBoost`

# Predicting probabilities

---

- Fraction of class in leaf.
- Without pruning: Always 100% certain!
- Even with pruning might be too certain.
- Evaluating and calibrating probabilities: Week 7

# Ensemble Models



# Poor man's ensembles

---

- Build different models
- Average the result
- Common in competitions: average the same gradient-boosting model trained with different random seeds.
- More models are better – if they are not correlated.
- Also works with neural networks
- You can average any models as long as they provide calibrated ("good") probabilities.
- Scikit-learn: VotingClassifier hard and soft voting

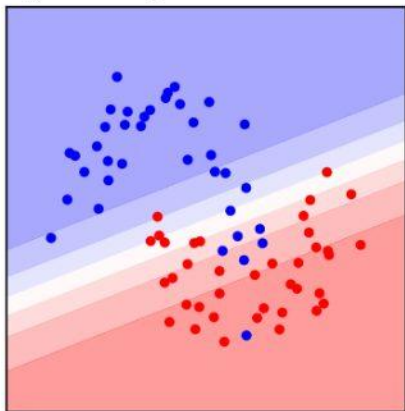


# VotingClassifier

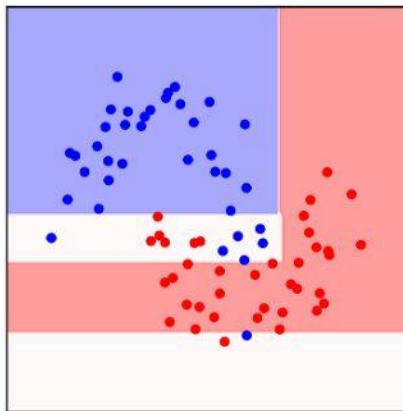
```
voting = VotingClassifier(  
    [('logreg', LogisticRegression(C=100)),  
    ('tree', DecisionTreeClassifier(max_depth=3, random_state=0))],  
    voting='soft')  
voting.fit(X_train, y_train)  
lr, tree = voting.estimators_  
tree.score(X_test, y_test), lr.score(X_test, y_test), voting.score(X_test, y_test)
```

0.80 0.84 0.88

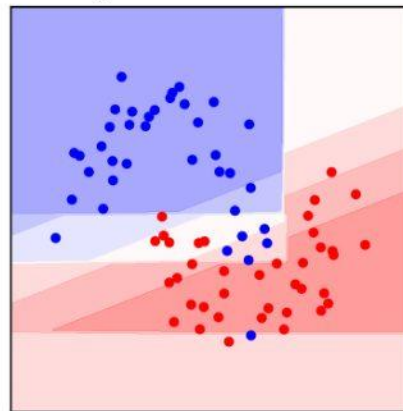
Logistic Regression acc=0.84



Decision Tree acc=0.80



Voting Classifier acc=0.88



# Stacking

```
from sklearn.ensemble import StackingClassifier
from sklearn.neighbors import KNeighborsClassifier

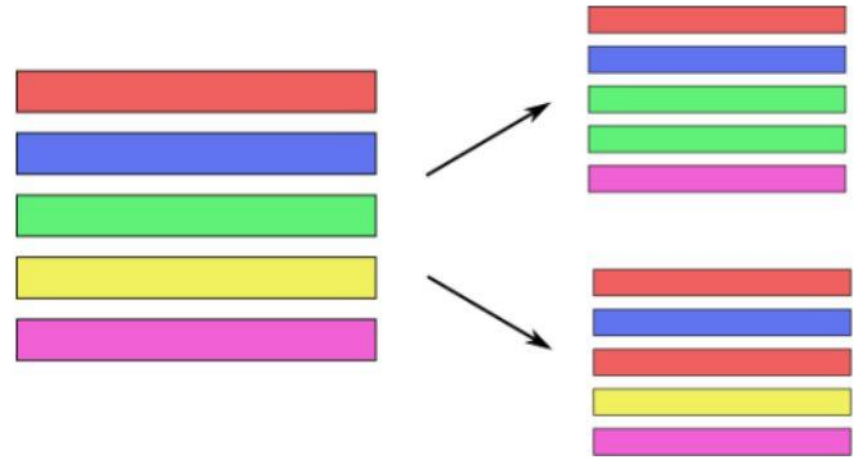
stack = StackingClassifier(
    estimators=[('logreg', LogisticRegression(C=100)),
                ('tree', DecisionTreeClassifier(max_depth=3, random_state=0)),
                ('knn', KNeighborsClassifier(n_neighbors=3))],
    final_estimator=LogisticRegression(),
    cv=5, stack_method='predict_proba')
stack.fit(X_train, y_train)
```

- Instead of averaging with fixed weights, **learn** how to combine the base models
- The final model is trained on **cross-validated** predictions of the base models

# Bagging (Bootstrap AGGregation)

---

- Generic way to build "slightly different" models
- BaggingClassifier, BaggingRegressor
- Draw bootstrap samples from dataset
- (as many as there are in the dataset, with repetition)

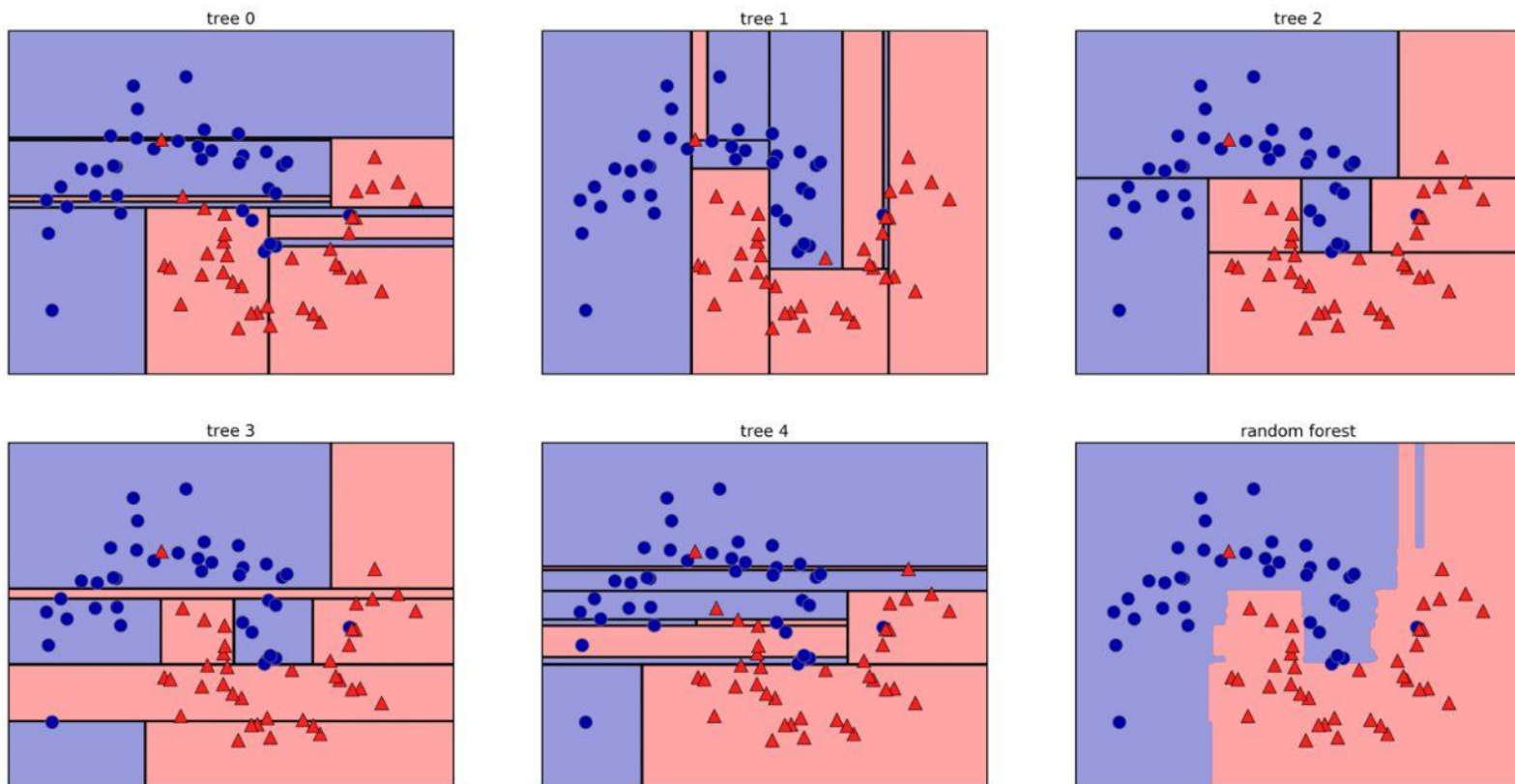


# Bias and Variance in Ensembles

---

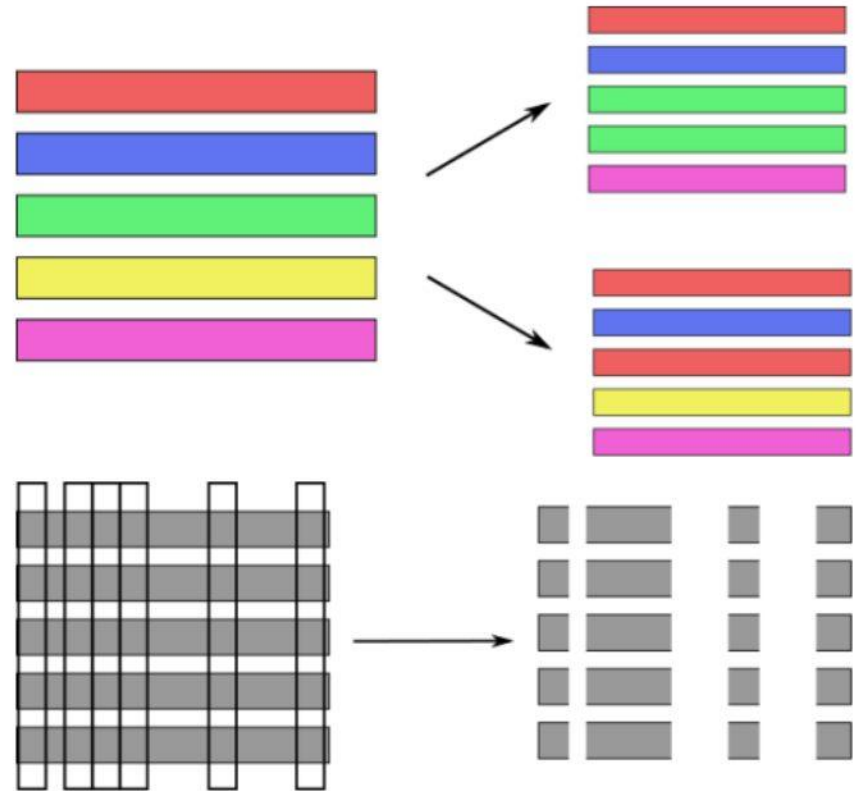
- Breiman showed that generalization depends on strength of the individual classifiers and (inversely) on their correlation
- Uncorrelating them might help, even at the expense of strength

# Random Forests



# Randomize in two ways

- For each tree:
  - Pick bootstrap sample of data
- For each split:
  - Pick random sample of features
- More trees are always better





# Tuning Random Forests

---

- Main parameter: `max_features`
  - default `"sqrt"` ( $\sqrt{n\_features}$ ) for classification
  - default `1.0` (all features) for regression, where smaller values often help
- `n_estimators`: default 100; more is better but with diminishing returns
- Prepruning might help, definitely helps with model size!
- `max_depth`, `max_leaf_nodes`, `min_samples_leaf` again

# Extremely Randomized Trees

---

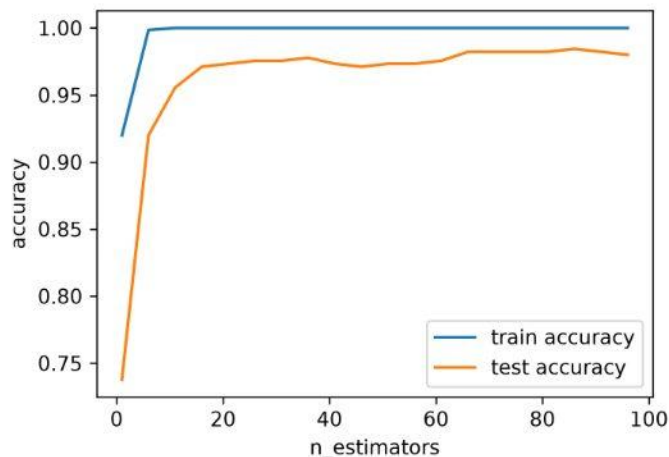
- More randomness!
- Randomly draw threshold for each feature!
- Doesn't use bootstrap
- Faster because no sorting / searching
- Can have smoother boundaries



# Warm-Starts

```
train_scores = []
test_scores = []

rf = RandomForestClassifier(warm_start=True, random_state=0)
estimator_range = range(1, 100, 5)
for n_estimators in estimator_range:
    rf.n_estimators = n_estimators
    rf.fit(X_train, y_train)
    train_scores.append(rf.score(X_train, y_train))
    test_scores.append(rf.score(X_test, y_test))
```

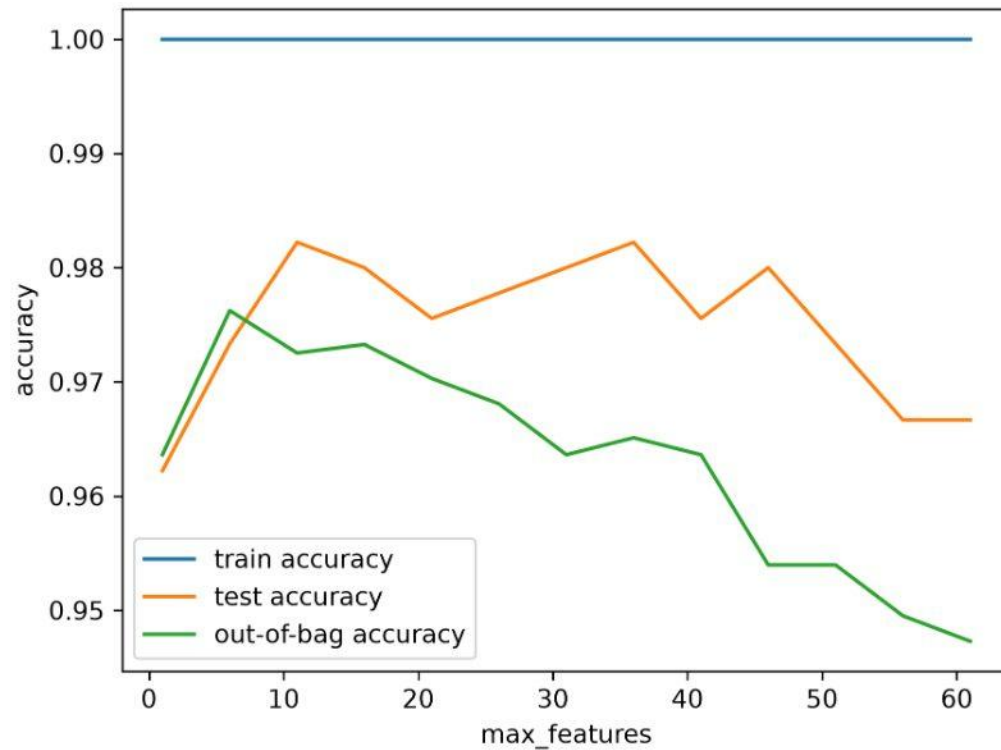


# Out-of-bag estimates

- Each tree only uses ~63% of data
- Can evaluate it on the rest!
- Make predictions for out-of-bag, average, score.
- Each prediction is an average over different subset of trees

```
train_scores = []
test_scores = []
oob_scores = []

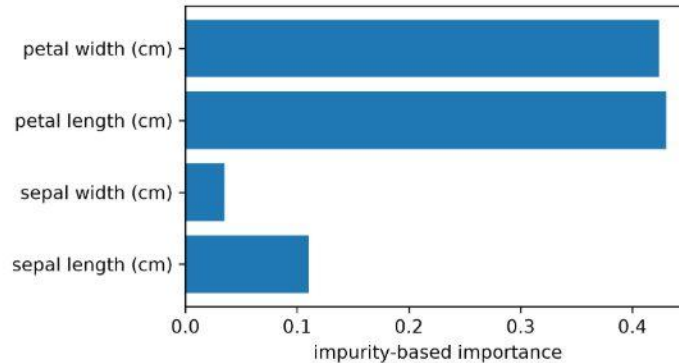
feature_range = range(1, 64, 5)
for max_features in feature_range:
    rf = RandomForestClassifier(max_features=max_features, oob_score=True,
                               n_estimators=200, random_state=0, n_jobs=-1)
    rf.fit(X_train, y_train)
    train_scores.append(rf.score(X_train, y_train))
    test_scores.append(rf.score(X_test, y_test))
    oob_scores.append(rf.oob_score_)
```



# Variable Importance

```
X_train, X_test, y_train, y_test = train_test_split(  
    iris.data, iris.target, stratify=iris.target, random_state=1)  
rf = RandomForestClassifier(random_state=0).fit(X_train, y_train)  
rf.feature_importances_
```

```
array([0.111, 0.035, 0.43 , 0.424])
```



# What goes wrong in practice

---

- **Extrapolation:** forecasting a trend with trees returns the last leaf value. Model differences or ratios instead.
- **Impurity importances reward noise:** fully grown trees split on nearly unique columns (IDs, survey weights). A random ID column ranks near the top in `feature_importances_` and has zero permutation importance on test data (see lab).
- **Default forests on noisy tables:** fully grown trees can lose to a tuned single tree; `min_samples_leaf` fixes it.
- **Tuning on the test set:** choosing `n_estimators` or the early-stopping point on the test set gives an optimistic score.
- **ID-like or leaky columns:** trees will find them; check what the top splits use.
- **Training accuracy that falls as boosting iterations increase:** numerical instability, not overfitting (e.g. a very rare class gives huge Newton leaf values). Fix with `l2_regularization` (e.g. 1.0), a smaller `learning_rate` or a larger `min_samples_leaf`.

# Try it 1: tree vs. forest (10 minutes)

---

`lab.ipynb`, section "Try it 1", Adult census data.

1. **First, write down your prediction:** does a single decision tree with CV-tuned `max_leaf_nodes` or a default `RandomForestClassifier` reach the higher test accuracy? By how much?
2. Complete the TODO cells and run them.
3. Refit the forest with `min_samples_leaf=5`. What changed, and why?



**Questions ?**



## Part 2: Gradient Boosting

# Boosting

---

$$f(x) = \sum_k g_k(x)$$

- Family of algorithms to create "strong" learner  $f$  from "weak" learners  $g_k$ .
- AdaBoost, GentleBoost, LogitBoost, ...

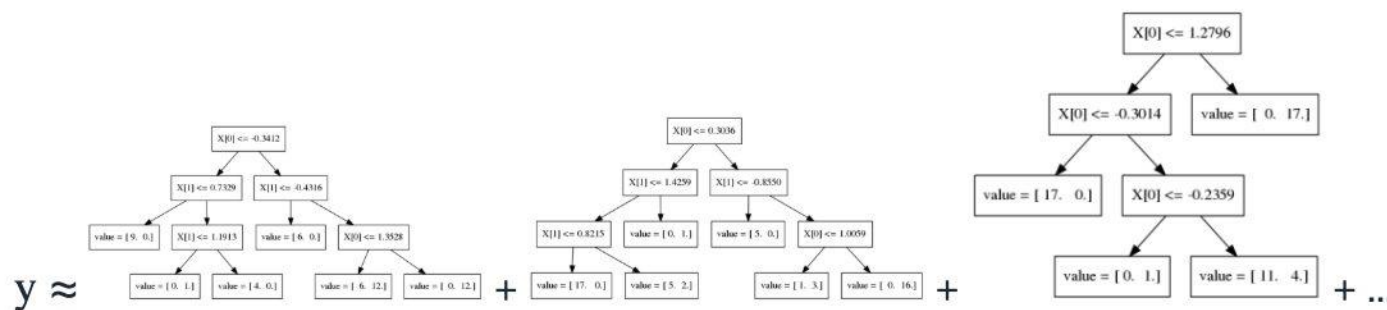
# Gradient Boosting

# Gradient Boosting Algorithm

$$f_1(x) \approx y$$

$$f_2(x) \approx y - f_1(x)$$

$$f_3(x) \approx y - f_1(x) - f_2(x)$$

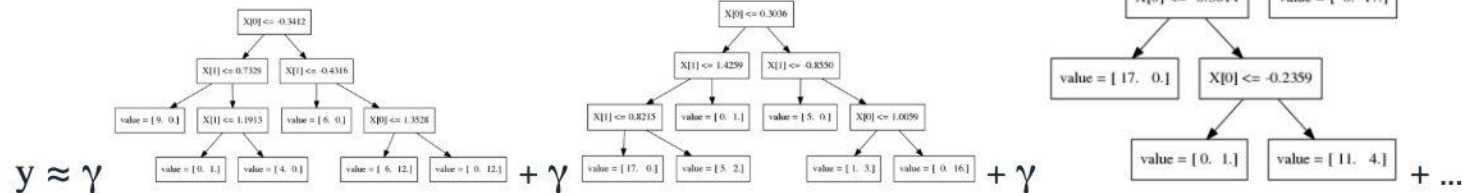


# Gradient Boosting Algorithm

$$f_1(x) \approx y$$

$$f_2(x) \approx y - \gamma f_1(x)$$

$$f_3(x) \approx y - \gamma f_1(x) - \gamma f_2(x)$$



$y \approx \gamma$

Learning rate  $\gamma$ , i.e. 0.1

# Gradient Boosting is Gradient Descent

## Linear regression

$$\begin{aligned} L(\mathbf{x}_i, y_i, \mathbf{w}, b) &= \sum_i (y_i - \hat{y}_i)^2 \\ &= \sum_i (y_i - \mathbf{w}^T \mathbf{x}_i - b)^2 \end{aligned}$$

optimize:

$$\min_{\mathbf{w} \in \mathbb{R}^p, b \in \mathbb{R}} \sum_{i=1}^n (y_i - \mathbf{w}^T \mathbf{x}_i - b)^2$$

gradient descent:

$$\mathbf{w}_{j+1} = \mathbf{w}_j - \gamma \frac{\partial L(\mathbf{x}_i, y_i, \mathbf{w}, b)}{\partial \mathbf{w}}$$

## Gradient Boosting

$$L(y_i, \hat{y}_i) = \sum_i (y_i - \hat{y}_i)^2$$

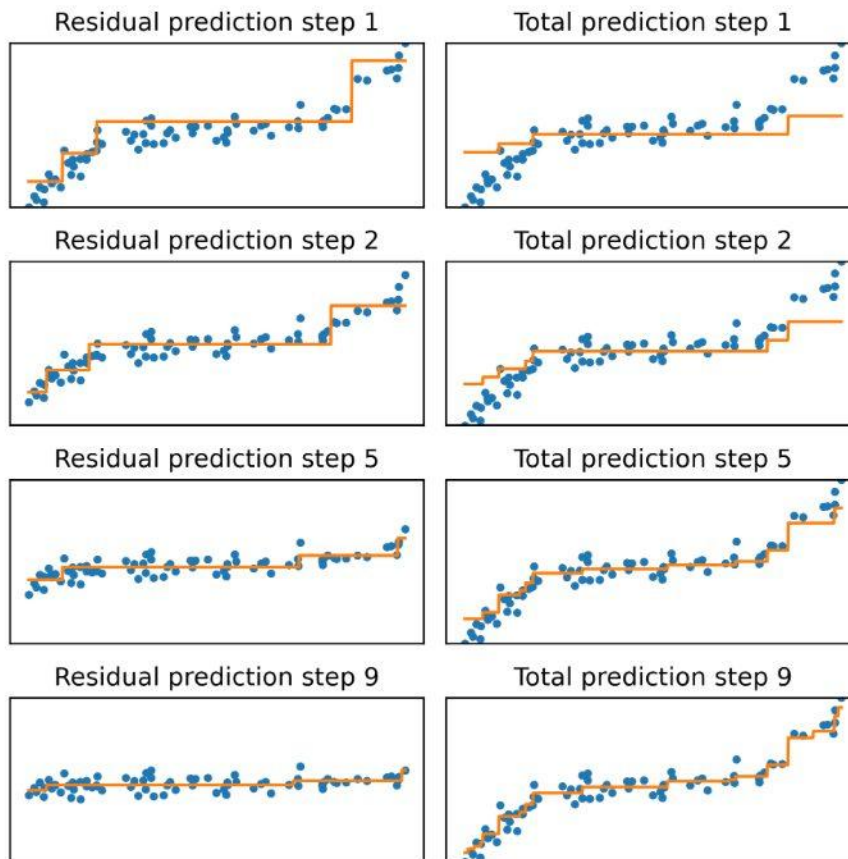
optimize:

$$\min_{\hat{\mathbf{y}} \in \mathbb{R}^n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

gradient descent:

$$\hat{y}_{j+1} = \hat{y}_j - \gamma \frac{\partial L(y_i, \hat{y}_i)}{\partial \hat{y}}$$

# GradientBoostingRegressor





# Gradient boosting for classification

---

Logistic regression:

$$\min_{\mathbf{w} \in \mathbb{R}^p, b \in \mathbb{R}} \sum_{i=1}^n \log(\exp(-y_i(\mathbf{w}^T \mathbf{x}_i + b)) + 1)$$

Gradient boosting:

$$\min_{\hat{\mathbf{y}} \in \mathbb{R}^n} \sum_{i=1}^n \log(\exp(-y_i \hat{y}_i) + 1)$$

# Multiclass Gradient Boosting

---

Multinomial logistic regression:

$$p(y = c|x) = \frac{e^{\mathbf{w}_c^T \mathbf{x} + b_c}}{\sum_{j=1}^k e^{\mathbf{w}_j^T \mathbf{x} + b_j}}$$

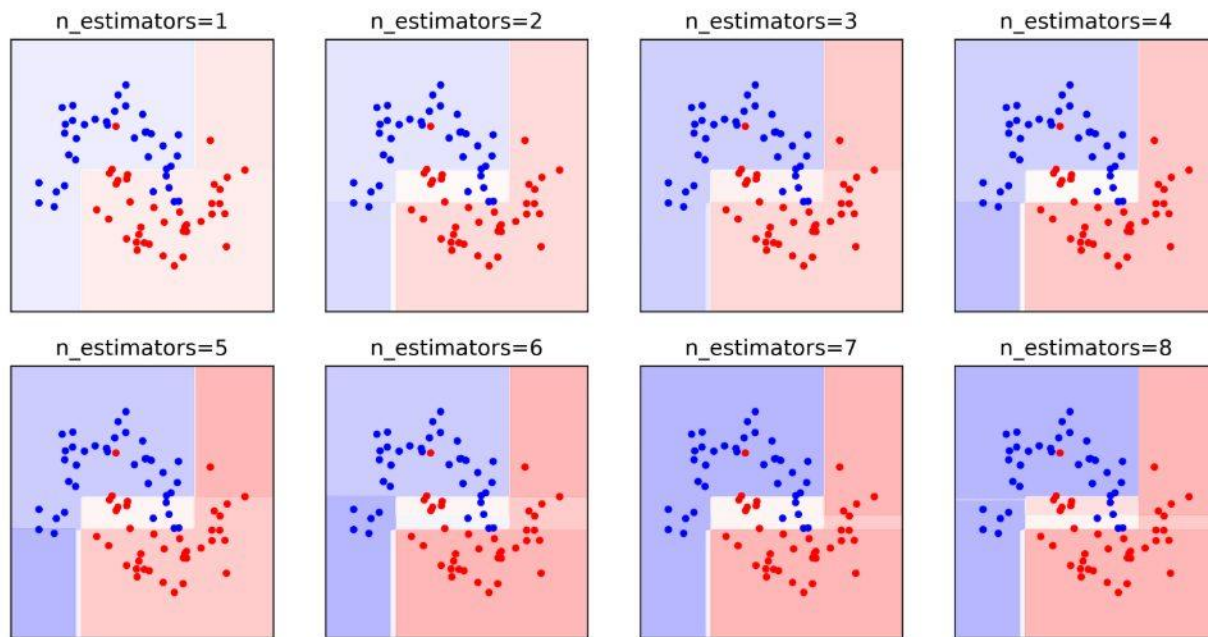
Multi-class gradient boosting:

$$p(y = c|x) = \frac{e^{y^{(c)}}}{\sum_{j=1}^k e^{y^{(j)}}}$$

One regression tree per class (per gradient step).

# GradientBoostingClassifier

GradientBoostingClassifier(max\_depth=2, learning\_rate=0.2)



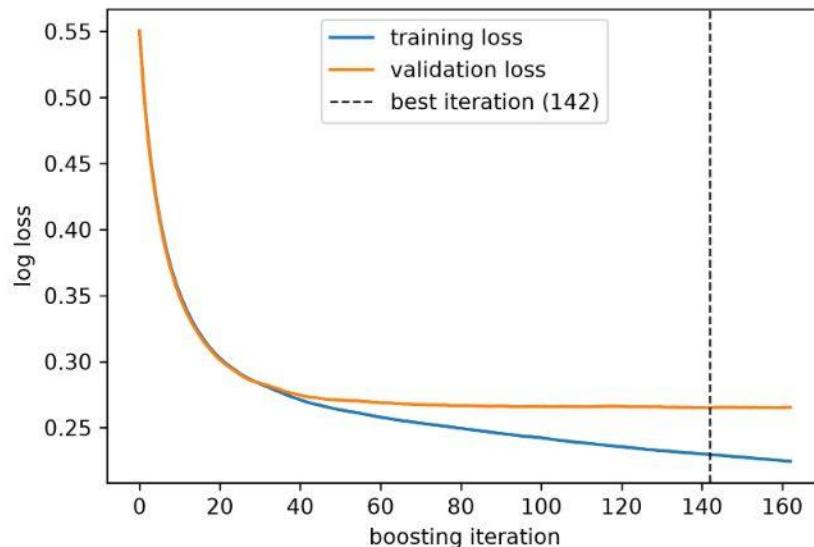
# Early stopping

- Adding trees can lead to overfitting
- Stop adding trees when the validation loss stops improving

two choices:

- pick number of trees and tune learning rate
- pick learning rate, use early stopping

```
hgb = HistGradientBoostingClassifier(  
    max_iter=2000, learning_rate=0.1,  
    early_stopping=True, validation_fraction=0.1,  
    n_iter_no_change=20)  
hgb.fit(X_train, y_train) # Adult data  
hgb.n_iter_                # 162
```

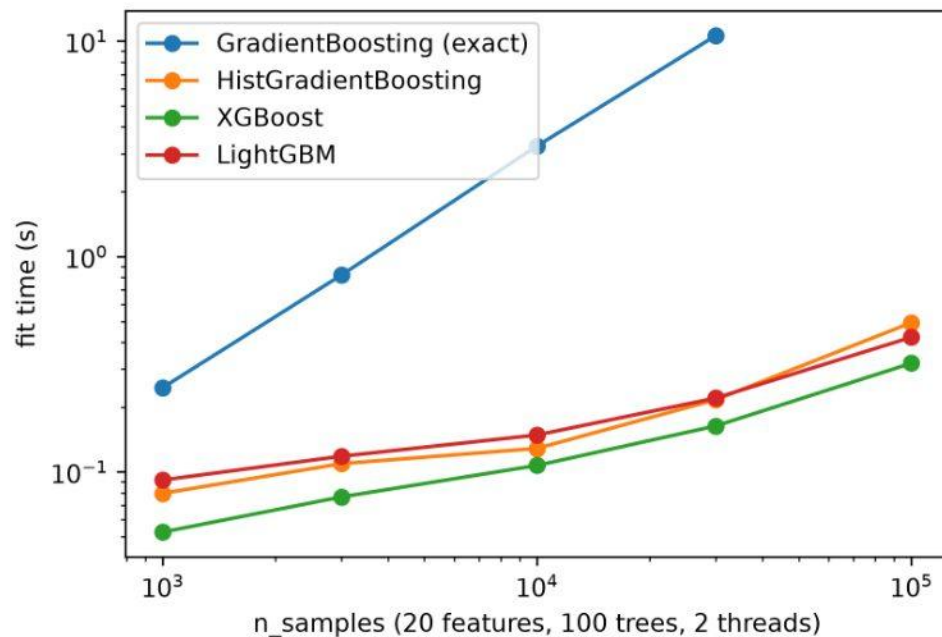


# Tuning of Gradient Boosting

---

- Typically strong pruning: `max_depth` 2-8 or `max_leaf_nodes` 15-63
- Pick learning rate and do early stopping
- Or pick `n_estimators`, tune learning rate (if not early stopping)
- `min_samples_leaf`, `l2_regularization`
- Column subsampling (`max_features`), row subsampling
- Tuning budget: random search or Bayesian optimization (Week 8)

# HistGradientBoostingClassifier





# GradientBoostingClassifier vs HistGradientBoostingClassifier

- exact splits, no binning
  - single core
  - sparse data support
  - no missing values
  - `n_iter_no_change` for early stopping (off by default)
- binning (up to 255 bins)
  - multicore (OpenMP)
  - dense input only
  - native missing values
  - native categorical features
  - monotonic and interaction constraints
  - early stopping on by default for  $n > 10,000$

# HistGradientBoosting in practice

```
from sklearn.datasets import fetch_openml
from sklearn.ensemble import HistGradientBoostingClassifier

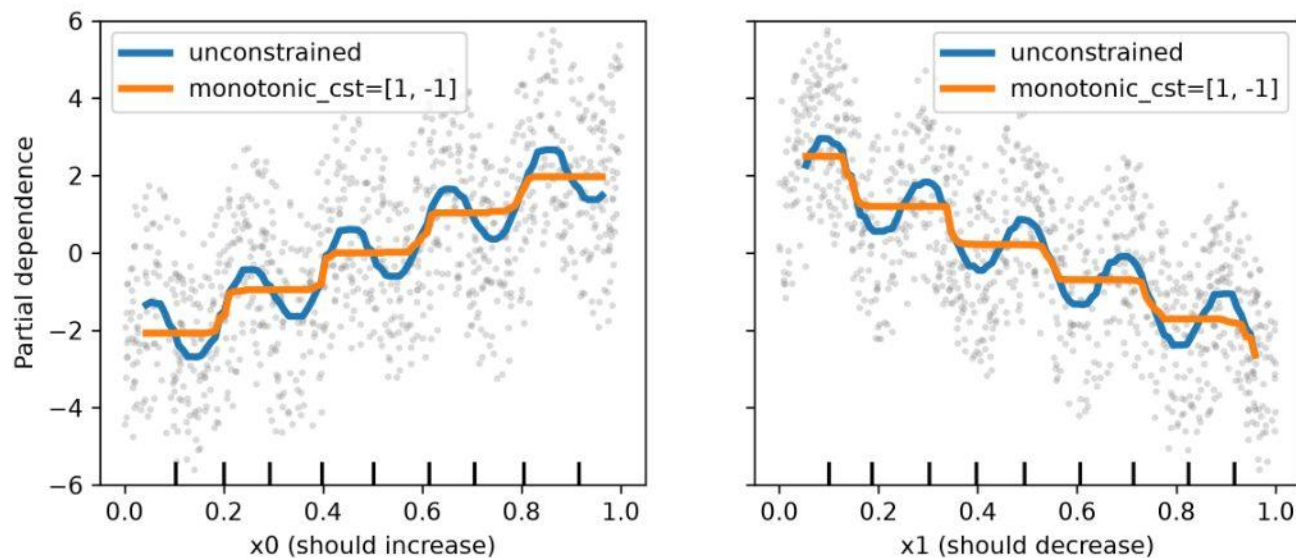
adult = fetch_openml("adult", version=2, as_frame=True)
X = adult.data # 8 of 14 columns have dtype "category"
y = (adult.target == ">50K").astype(int)
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, random_state=

hgb = HistGradientBoostingClassifier(
    categorical_features="from_dtype", # the default: use pandas category columns
    max_iter=1000, early_stopping=True, n_iter_no_change=20, random_state=0)
hgb.fit(X_train, y_train) # no encoder, no imputer
hgb.n_iter_, hgb.score(X_test, y_test) # (162, 0.872)
```

- Categories are split into subsets, ordered by gradient statistics
- Missing values: the learned best side at each split



# Monotonic constraints



```
HistGradientBoostingRegressor(monotonic_cst=[1, -1]) # or {"education-num": 1} fo
```

# XGBoost vs. LightGBM vs. CatBoost in practice

Adult census, 36,631 training / 12,211 test rows, near-default settings, one run on a laptop (see `lab.ipynb`):

| Model                    | Categoricals    | Test ROC AUC | Test accuracy | Fit time |
|--------------------------|-----------------|--------------|---------------|----------|
| RandomForest (300 trees) | ordinal-encoded | 0.916        | 0.866         | ~1 s     |
| HistGradientBoosting     | native          | 0.928        | 0.872         | < 1 s    |
| XGBoost                  | native          | 0.929        | 0.873         | < 1 s    |
| LightGBM                 | native          | 0.928        | 0.870         | ~1 s     |
| CatBoost                 | native          | 0.929        | 0.873         | 3-8 s    |

| HistGB     |                                | XGBoost                   | LightGBM                  | CatBoost                 |
|------------|--------------------------------|---------------------------|---------------------------|--------------------------|
| # trees    | <code>max_iter</code>          | <code>n_estimators</code> | <code>n_estimators</code> | <code>iterations</code>  |
| tree size  | <code>max_leaf_nodes</code>    | <code>max_depth</code>    | <code>num_leaves</code>   | <code>depth</code>       |
| L2 penalty | <code>l2_regularization</code> | <code>reg_lambda</code>   | <code>reg_lambda</code>   | <code>l2_leaf_reg</code> |

# Are these differences real?

The test set is a sample too. A **paired bootstrap** gives an interval for the difference between two models:

```
rng = np.random.default_rng(0)
diffs = []
for _ in range(2000):
    idx = rng.integers(0, len(y_test), len(y_test)) # resample test rows
    diffs.append(roc_auc_score(y_test[idx], p_a[idx])
                  - roc_auc_score(y_test[idx], p_b[idx]))
lo, hi = np.quantile(diffs, [0.025, 0.975])
```

- Resample the same rows for both models: the pairing cancels most of the shared noise
- If the interval contains 0, this test set cannot tell the two models apart
- Check this before you call a 0.001 AUC difference a win

# Why trees still win on tabular data

---

Grinsztajn, Oyallon & Varoquaux (2022), *Why do tree-based models still outperform deep learning on typical tabular data?*, NeurIPS Datasets & Benchmarks.

- 45 medium-sized tabular datasets (~10K samples), equal hyperparameter-search budgets
- Tree ensembles (XGBoost, gradient-boosted trees, random forests) beat deep tabular models on most datasets
- Three reasons, each tested by modifying the data:
  1. **Irregular target functions:** neural networks are biased towards smooth solutions
  2. **Uninformative features** hurt MLP-like networks much more than trees
  3. **Tabular data is not rotation invariant:** columns have meaning; trees exploit axis alignment, MLPs do not



# Gradient Boosting Advantages

---

- Very fast using HistGradientBoosting (or XGBoost, LightGBM)
- Small model size
- Typically more accurate than Random Forests
- Native missing values, categoricals, constraints
- "old" GradientBoosting in sklearn is comparatively slow

# Try it 2: a small competition (15 minutes)

---

lab.ipynb, section "Try it 2".

- Goal: highest **test ROC AUC** on Adult with `HistGradientBoostingClassifier`
- Choose hyperparameters with **5-fold CV on the training set only**; evaluate on the test set **once**
- Log every CV attempt to MLflow (experiment `week05-competition`)
- At the end: collect everyone's test score. How large a difference would convince you?

# Concluding tree-based models



# When to use tree-based models

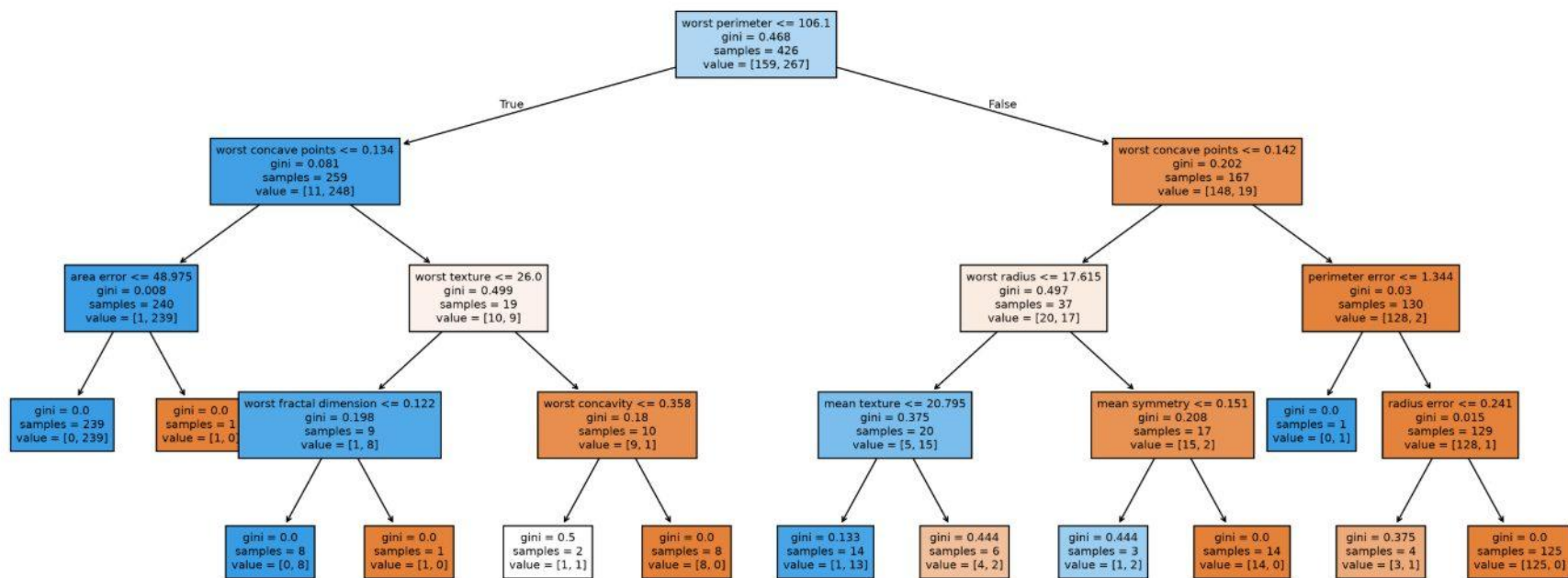
---

- Model non-linear relationships
- Don't care about scaling, little feature engineering needed
- Single tree: very interpretable (if small)
- Random forests very robust, good benchmark
- Gradient boosting often best performance with careful tuning
- Less suitable: extrapolation, very high-dimensional sparse data (e.g. text), images, audio

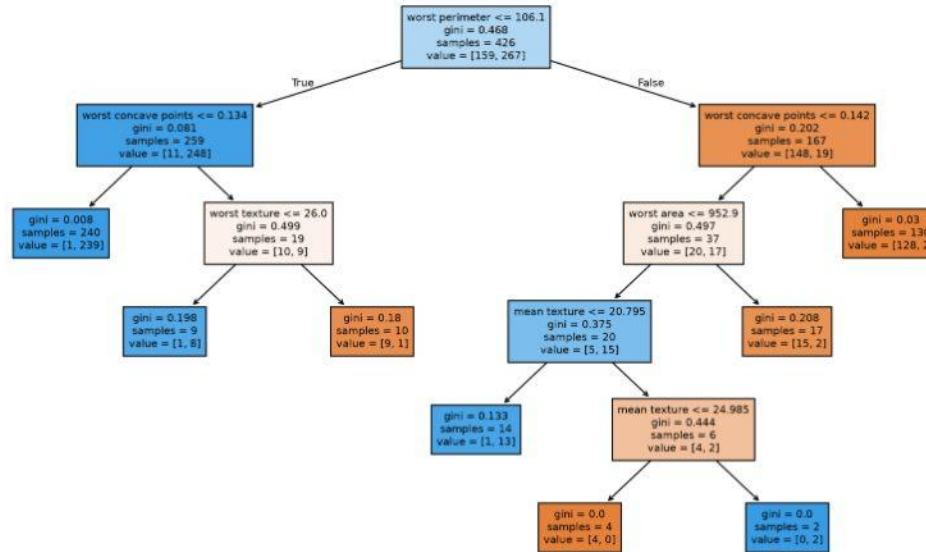
**Questions ?**

# Backup

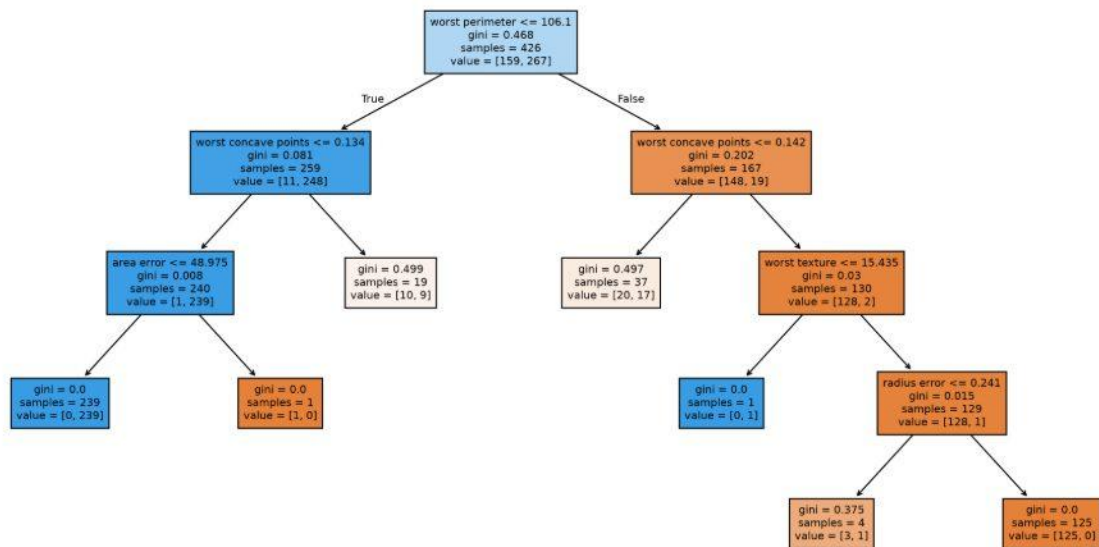
# max\_depth = 4



# max\_leaf\_nodes = 8



# min\_samples\_split = 50



# Cost Complexity Pruning

---

$$R_{\alpha}(T) = R(T) + \alpha|T|$$

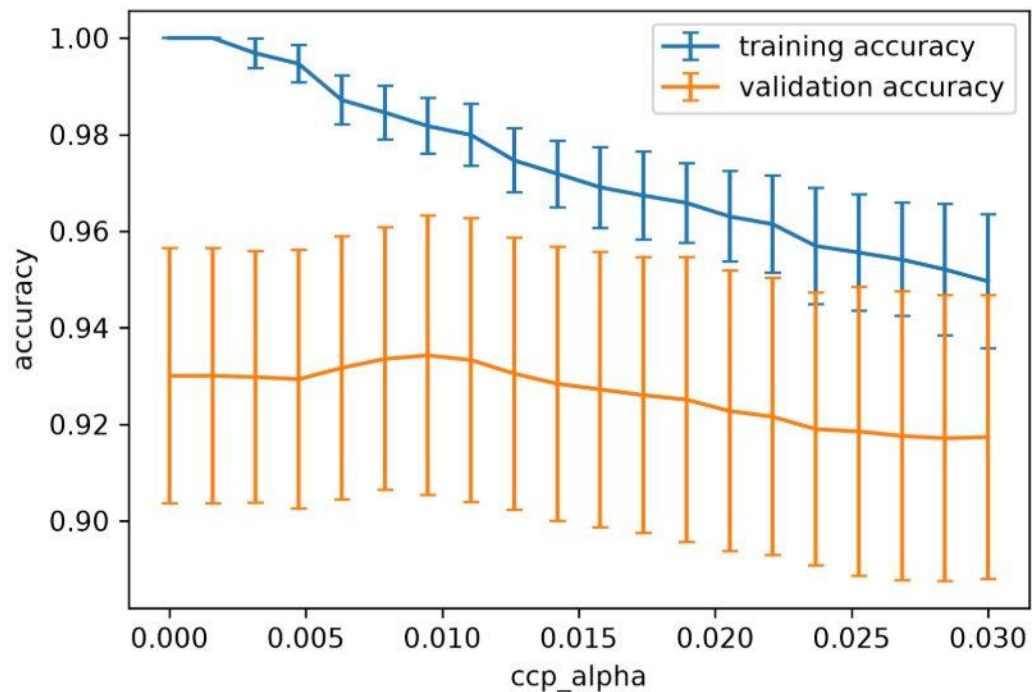
$R(T)$  is total leaf impurity

$|T|$  is number of leaf nodes

$\alpha$  is free parameter.

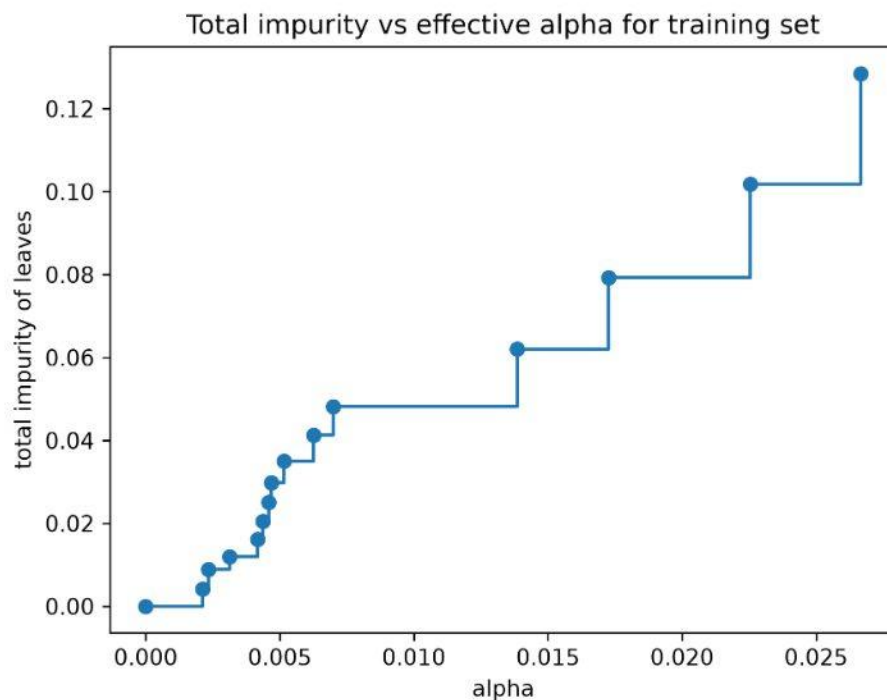


```
param_grid = {'ccp_alpha': np.linspace(0., 0.03, 20)}  
grid = GridSearchCV(DecisionTreeClassifier(random_state=0),  
                    param_grid=param_grid, cv=cv, return_train_score=True)  
grid.fit(X_train, y_train)
```



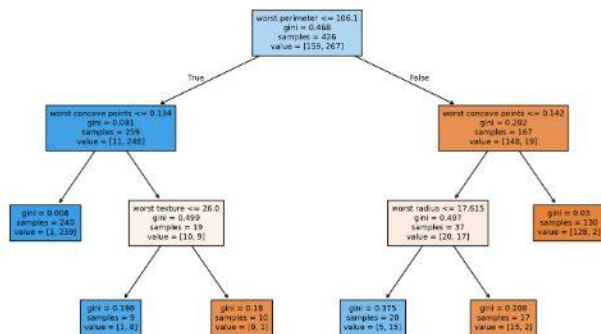
# Efficient pruning

```
clf = DecisionTreeClassifier(random_state=0)
path = clf.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas, impurities = path.ccp_alphas, path.impurities
```

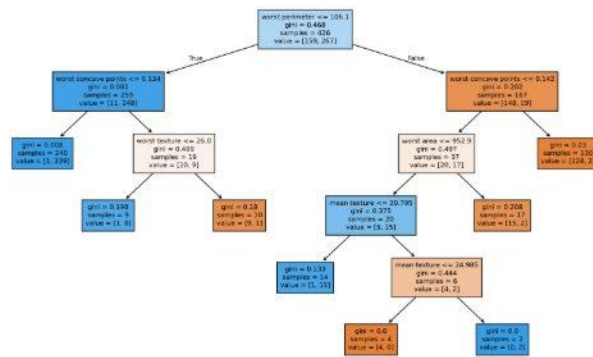


# Best post-pruned vs pre-pruned

## Cost-complexity pruning



## Max leaf nodes search



# Reminder: Gradient Descent

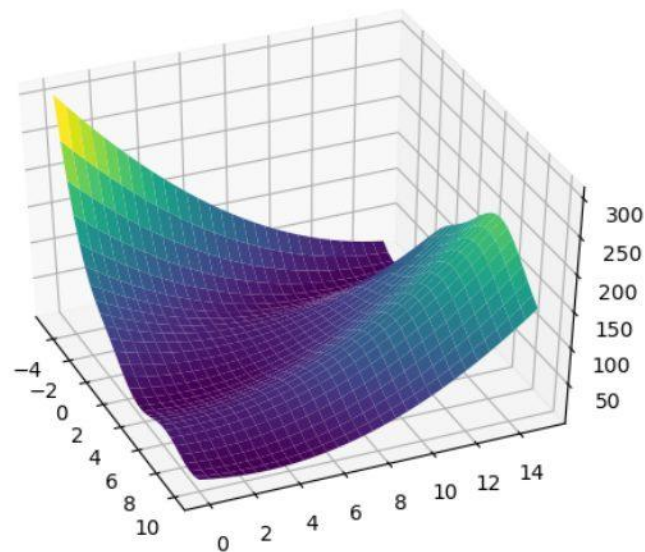
Want:

$$\arg \min_w F(w)$$

Initialize  $w_0$

$$w^{(i+1)} \leftarrow w^{(i)} - \eta_i \frac{d}{dw} F(w^{(i)})$$

Converges to local minimum



# Improvements:

## "extreme" gradient boosting

XGBoost: A Scalable Tree Boosting System, 2016

# Speeding up tree-building via binning

# Split finding is slow

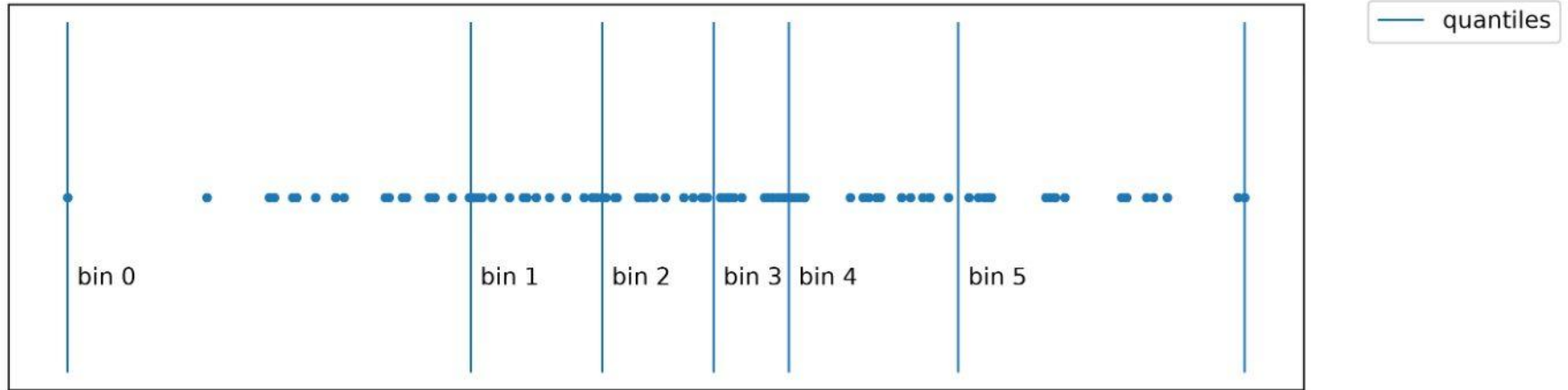
---

```
for feature in features:  
    for threshold in thresholds(f):  
        gain = compute_gain(feature, threshold)  
        if gain > best_gain:  
            best_split = (feature, threshold)
```

- Thresholds: all unique values of feature.
- Scanning thresholds = sorting:  $O(n \log n)$



# Binning features



# Binned split finding is fast

---

**Algorithm 1:** Exact Greedy Algorithm for Split Finding

---

**Input:**  $I$ , instance set of current node

**Input:**  $d$ , feature dimension

$gain \leftarrow 0$

$G \leftarrow \sum_{i \in I} g_i, H \leftarrow \sum_{i \in I} h_i$

**for**  $k = 1$  **to**  $m$  **do**

$G_L \leftarrow 0, H_L \leftarrow 0$

**for**  $j$  in sorted( $I$ , by  $\mathbf{x}_{jk}$ ) **do**

$G_L \leftarrow G_L + g_j, H_L \leftarrow H_L + h_j$

$G_R \leftarrow G - G_L, H_R \leftarrow H - H_L$

$score \leftarrow \max(score, \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda})$

**end**

**end**

**Output:** Split with max score

---

---

**Algorithm 2:** Approximate Algorithm for Split Finding

---

**for**  $k = 1$  **to**  $m$  **do**

    Propose  $S_k = \{s_{k1}, s_{k2}, \dots, s_{kl}\}$  by percentiles on feature  $k$ .

    Proposal can be done per tree (global), or per split(local).

**end**

**for**  $k = 1$  **to**  $m$  **do**

$G_{kv} \leftarrow \sum_{j \in \{j | s_{k,v} \geq \mathbf{x}_{jk} > s_{k,v-1}\}} g_j$

$H_{kv} \leftarrow \sum_{j \in \{j | s_{k,v} \geq \mathbf{x}_{jk} > s_{k,v-1}\}} h_j$

**end**

Follow same step as in previous section to find max score only among proposed splits.

---

# A better split criterion

---

- Include second order (hessian) and regularizer

$$\mathcal{L}_{split} = \frac{1}{2} \left[ \frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma$$

# XGBoost

```
from xgboost import XGBClassifier
xgb = XGBClassifier(tree_method="hist", enable_categorical=True)
xgb.fit(X_train, y_train)
xgb.score(X_test, y_test)
```

- Efficient implementation of gradient boosting
- L1 and L2 penalty on leaf weights, second-order split criterion
- Native missing values and categorical features
- GPU and distributed training
- Monotonicity and interaction constraints
- Supports sparse data
- Scikit-learn compatible interface

# LightGBM

```
from lightgbm import LGBMClassifier
lgbm = LGBMClassifier()
lgbm.fit(X_train, y_train)
lgbm.score(X_test, y_test)
```

- Leaf-wise (best-first) tree growth, controlled by `num_leaves`
- Native missing values and categorical variables (pandas category dtype)
- Gradient-based one-side sampling, exclusive feature bundling
- GPU and distributed training
- Monotonicity constraints
- Supports sparse data

# CatBoost

```
from catboost import CatBoostClassifier
catb = CatBoostClassifier(cat_features=cat_cols, verbose=0)
# CatBoost needs the categorical columns as strings without NaN
catb.fit(X_train_str, y_train)
catb.score(X_test_str, y_test)
```

- optimized for categorical variables
- uses one feature / threshold for all splits on a given level aka symmetric (oblivious) trees
- Symmetric trees are "different" but can be much faster to predict
- supports missing values
- GPU training
- monotonicity constraints
- ordered target statistics (a leakage-free version of target encoding) for categorical variables
- good results with default parameters

# Relation to Nearest Neighbors

---

- Predict average of neighbors – either by  $k$ , by epsilon ball or by leaf.
- Trees are much faster to predict.
- Both can't extrapolate

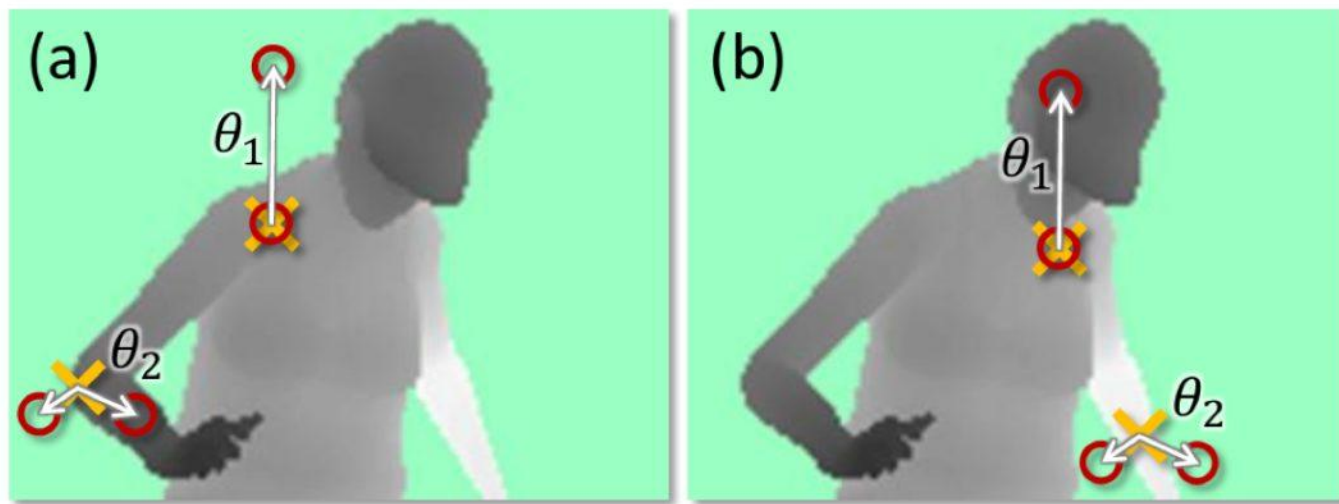


# Conditional Inference Trees

---

- Select "best" split with correcting for multiple-hypothesis testing.
- More "fair" to categorical variables.
- Available in R (`party`), not in scikit-learn

# Different splitting methods

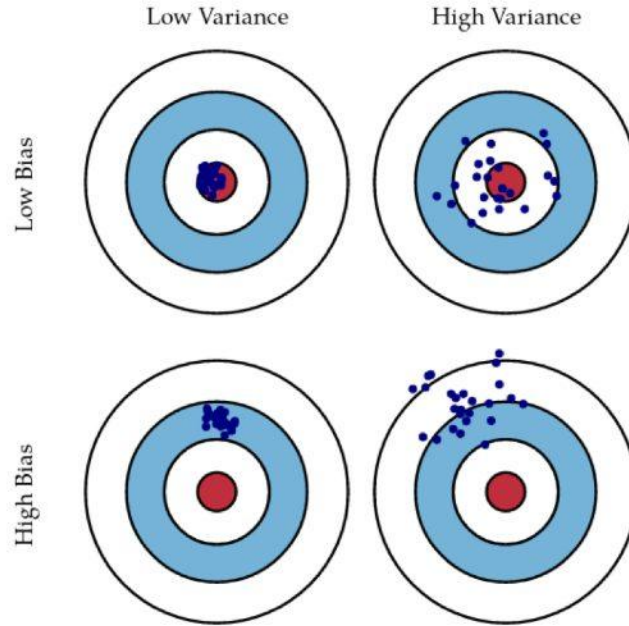


(taken from Shotton et. al. Real-Time Human Pose Recognition ..)

- Could use anything as split candidate!
- Linear models used if extrapolation is needed.
- Computer vision: pixel comparisons
- Kinect (first generation): depth comparison

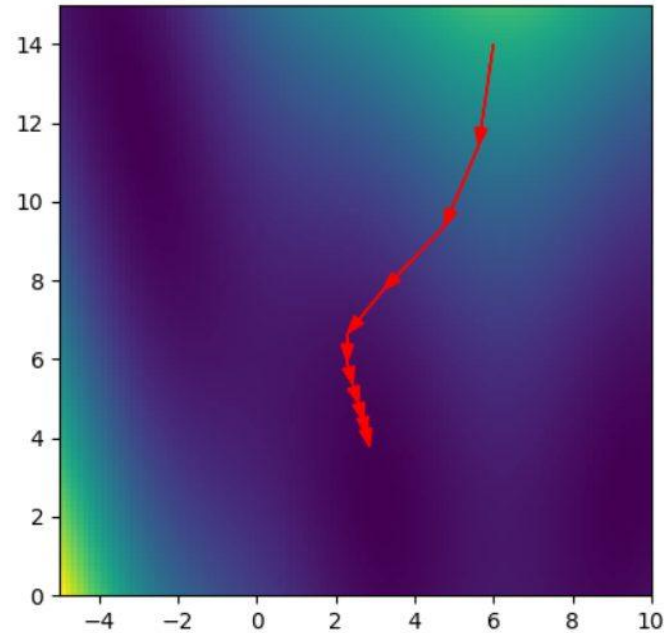
# Bias and Variance

---



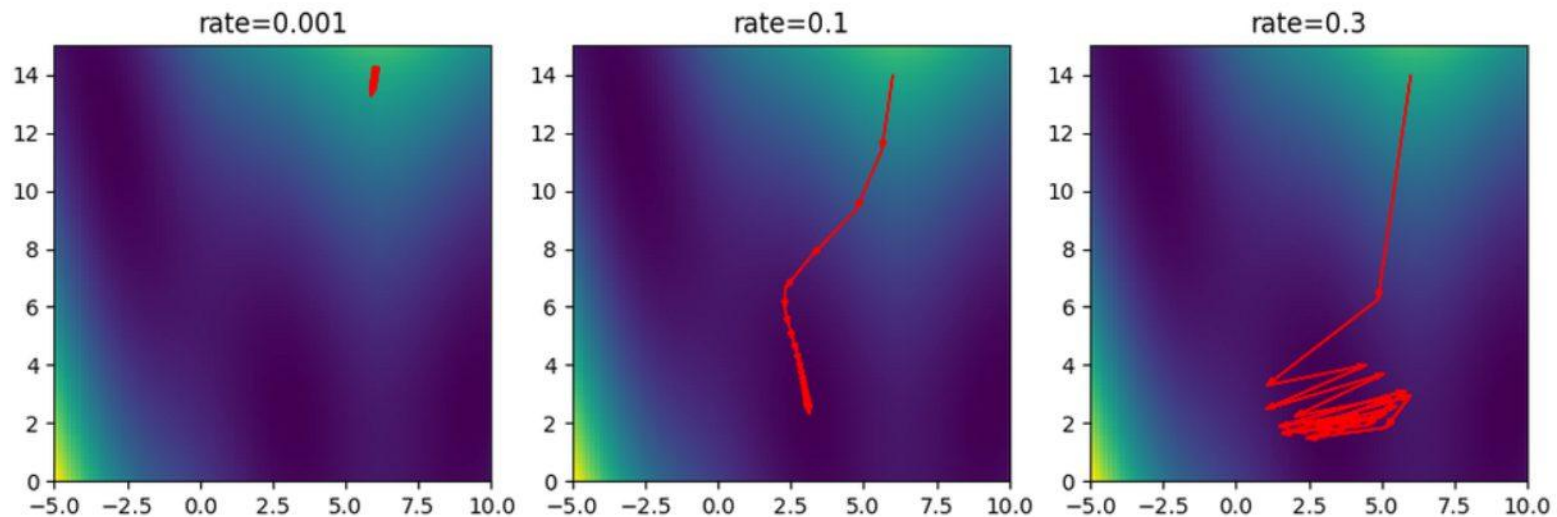
# Reminder: Gradient Descent

---



$$\mathbf{w}^{(i+1)} \leftarrow \mathbf{w}^{(i)} - \eta_i \frac{d}{d\mathbf{w}} F(\mathbf{w}^{(i)})$$

# Pick a learning rate



$$w^{(i+1)} \leftarrow w^{(i)} - \eta_i \frac{d}{dw} F(w^{(i)})$$

# Binning features

---

## Original

```
[[5. , 2. , 3.5, 1. ],  
 [4.9, 3. , 1.4, 0.2],  
 [4.4, 2.9, 1.4, 0.2],  
 [5. , 2.3, 3.3, 1. ],  
 [4.9, 2.5, 4.5, 1.7],  
 [6.3, 2.5, 5. , 1.9],  
 [6.3, 2.3, 4.4, 1.3],  
 [5. , 3.5, 1.3, 0.3],  
 [6.1, 2.8, 4.7, 1.2],  
 [5. , 3.5, 1.6, 0.6]])
```

## Binned

```
[[1, 0, 1, 1],  
 [0, 2, 0, 1],  
 [0, 1, 0, 1],  
 [1, 0, 1, 1],  
 [0, 0, 2, 3],  
 [3, 0, 3, 4],  
 [3, 0, 2, 2],  
 [1, 4, 0, 1],  
 [3, 1, 3, 2],  
 [1, 4, 1, 1]])
```

# Aggressive sub-sampling

---

According to user feedback, using column sub-sampling prevents overfitting even more so than the traditional row sub-sampling. (XGBoost: A Scalable Tree Boosting System, 2016)